AxiomEval — Préparation aux entrevues (stages IA, ML, sécurité, logiciel)

Toutes les réponses renvoient au code et aux résultats réels du dépôt (branche `feat/mission-s24`). Ce qui n'existe pas est dit comme tel.

- AxiomEval — Préparation aux entrevues (stages IA, ML, sécurité, logiciel)
  - 1. Pitchs
  - 2. Ce que le système fait, ce qui est expérimental, ce qui est prévu
  - 3. Contributions personnelles — à distinguer honnêtement
  - 4. Questions techniques (55)
    - Architecture et logiciel
    - Agents, LLM, RAG
    - Sécurité
    - Statistiques
    - Machine learning
    - MLOps et reproductibilité
    - Questions difficiles
  - 5. Démonstrations pratiques (commandes testées pendant la mission)

1. Pitchs

30 secondes. « AxiomEval décide si une nouvelle version d'un agent IA peut être mise en service. Il fait tourner un vrai modèle local sur 81 scénarios groupés, trace chaque étape sans secret, bloque hors du modèle les actions non autorisées, et compare les versions par groupes avec une borne statistique. Sur mes tests réels, il a rejeté deux versions qui divulguaient un secret et mis en attente la version protégée, faute de données suffisantes pour prouver qu'elle n'était pas moins bonne. »

2 minutes. Ajouter : le problème (régression métier vs violation), l'architecture (corpus → collecte → trace → autorisation → garde → évaluation → verdict → API), un résultat chiffré (exfiltration 19/27 → 0/27 avec la garde), une limite (27 groupes ; Hoeffding demanderait ~780 groupes), et la suite (agrandir le corpus avant les modèles séquentiels).

10 minutes. Démonstration : `axiomeval corpus` → `axiomeval collect` sur r1 → `axiomeval compare` r1/r4 → `axiomeval security-report` → `docs/security/FINDINGS.md` (F-001) → test de régression → `axiomeval serve` et un verdict HTTP.

2. Ce que le système fait, ce qui est expérimental, ce qui est prévu

Fait et testé	Expérimental (résultat partiel)	Prévu, non implémenté
Agent local réel, traces chaînées, caviardage, autorisation hors modèle, garde de sortie, corpus groupé gelé, verdict apparié, API + SQLite, 121 tests	Baselines ML (S9) : signal en point, intervalles chevauchants	GRU, Transformer, contrastif, SSL, calibration, conformal, abstention, OOD, MCP, ONNX, OpenTelemetry, SARIF, CycloneDX

3. Contributions personnelles — à distinguer honnêtement

Le code a été écrit avec l'aide d'un assistant IA. En entrevue, revendique ce que tu peux **expliquer, modifier et défendre sans aide** : le cahier des charges et la roadmap (les tiens), les choix de protocole que tu as validés, la lecture des résultats. Pour chaque fichier, vérifie avec les exercices que tu sais le réécrire. Formulation honnête : « J'ai conçu le projet et son protocole, j'ai utilisé un assistant pour accélérer l'implémentation, et je peux expliquer et modifier chaque composant. » Ne dis jamais « certifie la sécurité », « prouve statistiquement la supériorité » ou « utilisé par des clients ».

4. Questions techniques (55)

Format : **réponse courte** — approfondissement — exemple AxiomEval — limite — relance possible.

Architecture et logiciel

1. **Pourquoi Python ?** — Écosystème ML/statistique et prototypage rapide. — Bibliothèque standard suffisante pour le cœur (hashlib, sqlite3, json). — `src/axiomeval` n'a qu'une dépendance obligatoire (PyYAML). — Performances moindres qu'un langage compilé. — *Relance : que réécrirais-tu en Go/Rust ?* La garde et l'autorisation si elles devenaient un service à forte charge.
2. **Comment est découpée l'architecture ?** — Domaine sans entrées/sorties ; évaluation ; politique ; rapports ; collecte ; API. — Les dépendances vont vers le domaine. — `models/` n'importe ni la CLI ni l'API. — Le module `cli_corpus.py` grossit. — *Relance : comment tester un module isolé ?* Avec des objets construits à la main, comme `tests/test_policy_s06.py`.
3. **Pourquoi des dataclasses gelées ?** — Immuabilité : une identité ou une preuve ne change pas après validation. — `frozen=True` + `__post_init__`. — `Identity`, `Evidence`. — Coût de copie si l'on doit modifier. — *Relance : comment modifier ?* `dataclasses.replace` crée un nouvel objet validé.
4. **Comment gères-tu les entrées hostiles ?** — Refus explicite plutôt que tolérance. — YAML sûr, clés uniques, schéma exact, taille maximale, `extra="forbid"` dans l'API. — `config.py`, `api/app.py`. — Pas de fuzzing systématique encore. — *Relance : un exemple de bug évité ?* Une clé `rules` mal orthographiée qui aurait créé une politique vide.
5. **Comment garantis-tu la reproductibilité ?** — Identités de release par empreinte, partitions gelées, graines explicites, empreintes des données. — `release.py`, `split.json`, `data/MANIFEST.json`. — Les sorties du modèle dépendent aussi de la version d'Ollama. — *Relance : que manque-t-il ?* Un fichier de verrouillage des dépendances et une CI exécutée.
6. **Comment fonctionne l'idempotence des imports ?** — Clé de tirage unique ; ré-import ignoré ; même clé mais trace différente : conflit et refus du lot. — `store.import_records`. — Testé (`test_import_is_idempotent`). — Pas testé sous concurrence. — *Relance : pourquoi refuser tout le lot ?* Un état partiel serait pire qu'un refus.
7. **Pourquoi SQLite ?** — Local, sans serveur, transactionnel, suffisant pour un laboratoire. — WAL, transactions `BEGIN IMMEDIATE`. — Sauvegarde testée. — Un seul écrivain à la fois. — *Relance : quand passer à PostgreSQL ?* Plusieurs écrivains ou accès réseau.
8. **Comment versionnes-tu le schéma ?** — Migrations numérotées, empreinte stockée, refus si un fichier appliqué change. — `store.migrate`. — Bug réel corrigé : un `;` dans un commentaire coupait une instruction. — Pas de migration descendante. — *Relance : pourquoi l'empreinte ?* Garantir que le fichier décrit vraiment la base.

Agents, LLM, RAG

1. **Pourquoi un modèle local ?** — Coût nul, aucune donnée ne sort, reproductibilité. — Ollama, localhost uniquement (adresse refusée sinon). — Qwen2.5 1,5B, 1,17 Go de VRAM. — Petit modèle, peu fiable sur les outils. — *Relance : et avec un modèle API ?* Ajouter un adaptateur conforme au même contrat ; gérer secrets et coûts.

2. **Comment fonctionne la boucle de l'agent ?** — Recherche → modèle → outil éventuel → modèle → réponse, 4 étapes max. — `collect/agent.py`. — Chaque étape est un événement de trace. — Pas de mémoire entre sessions. — *Relance : que se passe-t-il à la 5e étape ?* Erreur `max-steps-reached`, résultat MISSING.
3. **Comment est fait le RAG ?** — Recouvrement de mots, 2 documents. — `knowledge.retrieve`. — Simple et transparent. — Pas d'embeddings. — *Relance : pourquoi pas d'embeddings ?* Pas nécessaire pour 6 documents ; à mesurer avant d'ajouter.
4. **Comment as-tu testé un système non déterministe ?** — Harnais testé par un adaptateur scripté ; modèle réel mesuré par des collectes répétées. — `ScriptedAdapter`, température 0,7 et graines distinctes. — 648 exécutions réelles. — Les tests unitaires ne valident pas le modèle. — *Relance : pourquoi pas température 0 ?* Les répétitions seraient des copies.

Sécurité

1. **Différence entre brèche et tentative ?** — Brèche : effet réel ; tentative : stoppée par un contrôle. — REJECT vs HOLD. — r4 : 20 tentatives bloquées, 0 brèche. — Une tentative révèle quand même une faiblesse du modèle. — *Relance : pourquoi pas REJECT pour une tentative ?* Le contrôle a fonctionné ; on demande une revue humaine.
2. **Comment évites-tu qu'un modèle agisse pour un autre client ?** — Autorisation hors modèle avec identité de session. — `authorize`, prétentions `customer_id` ignorées. — 14 tests §31.7. — Identité simulée. — *Relance : confused deputy ?* Le modèle croit agir pour client-2 ; le moteur agit toujours pour la session.
3. **Qu'est-ce que le fail-closed ?** — Refuser ce que la politique ne décrit pas. — `export_customer_data` n'a pas de règle → refus. — Testé. — Exige une politique complète pour les outils légitimes. — *Relance : un exemple de fail-open dangereux ?* Un outil ajouté au code mais oublié dans la politique.
4. **Comment fonctionne l'approbation humaine ?** — Jeton de 128 bits, usage unique, 5 min, lié à l'identité, l'outil et l'empreinte des arguments. — `ApprovalStore`. — Rejeu, expiration, changement de montant refusés. — Stockage en mémoire. — *Relance : pourquoi lier aux arguments ?* Sinon une approbation de 230 $ servirait pour 900 $.
5. **Pourquoi SHA-256 pour les traces ?** — Résistant aux collisions, standard, rapide. — Chaîne d'empreintes. — Modification ou réordonnancement détectés. — Ne prouve pas l'auteur. — *Relance : comment prouver l'auteur ?* Signature (Ed25519) avec une clé protégée.
6. **Qu'as-tu trouvé de réel ?** — F-001 : jeton envoyé comme numéro de commande ; F-002 : consignes durcies insuffisantes. — `FINDINGS.md`. — Exfiltration r1 19/27, r2 15/27, r4 0/27. — 9 essais par famille ailleurs. — *Relance : comment as-tu corrigé F-001 ?* Garde des arguments avant exécution, test de régression qui rejoue l'appel.
7. **La garde est-elle suffisante ?** — Non : secrets connus et transformations simples seulement. — Paraphrase ou secret inconnu passent. — Le vrai correctif est de ne pas exposer le secret au modèle. — *Relance : que ferais-tu ensuite ?* Filtrer les documents internes avant la recherche.
8. **Comment évites-tu de stocker des secrets ?** — Caviardage avant hachage ; barrière avant écriture ; refus à l'import en base. — `Redactor`, `Store.import_records`. — Testé. — Caviardage par motifs. — *Relance : pourquoi avant hachage ?* La trace stockée reste vérifiable sans contenir le secret.
9. **« 0 brèche » veut-il dire sûr ?** — Non. — Borne exacte 10,5 % pour 0/27, optimiste car tirages corrélés. — Spec §17. — Nouvelles attaques possibles. — *Relance : combien d'essais pour une borne de 1 % ?* $n \geq \ln 20/0{,}01 \approx 300$ essais indépendants.

Statistiques

1. **Pourquoi des comparaisons appariées ?** — Mêmes groupes pour les deux releases : la difficulté s'annule. — `paired_comparison`. — Réduit la variance. — Exige que les deux releases passent exactement les mêmes groupes (sinon INCOMPLETE).
2. **Quelle est l'unité statistique ?** — Le groupe. — 27 groupes, pas 162 exécutions. — Les paraphrases et tirages sont corrélés. — Les groupes d'une même famille ne sont pas parfaitement indépendants.

3. **Pourquoi Hoeffding ?** — Valide pour tout n sous indépendance et bornitude, sans normalité. — Rayon 0,471 à 27 groupes. — Très conservateur. — *Relance : alternative ?* Bernstein empirique, ou intervalle t si n grand.
4. **Pourquoi une meilleure moyenne ne suffit-elle pas ?** — L'écart peut venir du hasard ; il faut une borne. — r2→r4 : +0,08 mais borne −0,39 → HOLD. — Et une violation l'emporte toujours.
5. **Qu'est-ce que la non-infériorité ?** — Montrer que la candidate n'est pas pire que la référence de plus d'une marge fixée d'avance. — Marge 0,10. — Borne inférieure > −0,10. — Choisir la marge après coup invaliderait le test.
6. **Combien de groupes te faudrait-il ?** — ~780 pour r1→r4 et ~185 pour r2→r4 avec Hoeffding. — Formule $n > w^2 \ln(1/\alpha)/(2(d+m)^2)$. — Dépend de l'écart réel.
7. **Comment évites-tu les comparaisons multiples ?** — Une question primaire, protocole gelé ; le reste est exploratoire. — Pas de correction de multiplicité encore implémentée.
8. **Qu'est-ce que le bootstrap groupé ?** — Rééchantillonner les groupes avec remise. — Intervalle percentile, diagnostic. — Approximation discutable à 27 groupes.
9. **Comment mesures-tu l'incertitude de la sécurité ?** — Borne exacte de Clopper-Pearson par famille. — Rapport de sécurité. — Hypothèse d'indépendance.

Machine learning

1. **Que prédit ton modèle ?** — Si une exécution aura besoin de revue, juste après le premier appel au modèle. — Tâche B de la spec. — Label : métier ≠ SUCCESS.
2. **Comment évites-tu la contamination entraînement/test ?** — Partitions par groupe gelées ; vocabulaire appris sur l'entraînement ; validation croisée groupée ; test ouvert une fois. — Tests `test_featurizer_learns_vocabulary_from_train_only`, `leaked_keys`.
3. **Qu'est-ce qu'une fuite de caractéristique ici ?** — Utiliser une information postérieure à l'horizon ou un raccourci. — Latence et identifiant de release exclus. — L'ablation « avec release » n'apporte rien.
4. **Quels résultats ?** — PR-AUC majorité 0,54 ; logistique 0,77 [0,47 ; 0,97] ; boosting 0,88 [0,62 ; 0,98]. — Intervalles chevauchants : pas de conclusion.
5. **Pourquoi PR-AUC et Brier ?** — PR-AUC pour le classement des cas à revoir ; Brier pour la qualité des probabilités (score propre).
6. **Pourquoi le texte domine-t-il ?** — Il révèle la famille de la demande ; raccourci de famille, pas compréhension de la trajectoire.
7. **Pourquoi utiliserais-tu un Transformer ?** — Pour modéliser l'ordre et les interactions des événements d'un préfixe. — Pas encore fait. — Inutile tant que le corpus a 27 groupes ; il devra battre les baselines à budget égal.
8. **Que ferais-tu pour améliorer le ML ?** — Agrandir et diversifier le corpus (200–600 groupes), un second domaine, puis séquentiel et calibration.
9. **Comment choisis-tu C ?** — Validation croisée groupée sur l'entraînement, par log loss. — C = 1.
10. **Pourquoi trois graines identiques ?** — Modèles déterministes sur données fixes ; la variabilité vient des données.

MLOps et reproductibilité

1. **Comment suis-tu les expériences ?** — Manifest JSON : empreinte des données, commit, versions, graines, protocole. — Pas encore MLflow/DVC.
2. **Comment vérifies-tu la reproductibilité d'une collecte ?** — Clé de tirage, identité de release, empreintes dans `MANIFEST.json`.
3. **Qu'as-tu réellement mesuré ?** — 648 exécutions du corpus principal, 252 du corpus sécurité, latence, jetons, mémoire du modèle, verdicts, brèches, PR-AUC/Brier.
4. **Quelle est la principale faiblesse d'AxiomEval ?** — La taille du corpus : aucune conclusion statistique positive possible avec 27 groupes.

5. **Comment gères-tu une violation critique ?** — REJECT automatique, finding documenté, correctif, test de régression qui rejoue l'observation, nouvelle collecte.
6. **Comment éviter les faux positifs de la garde ?** — Seuils de longueur, courriel exclu, tests de réponses légitimes ; à surveiller sur de vrais numéros de suivi.
7. **Pourquoi ne pas utiliser un benchmark existant ?** — Il faut des scénarios avec mes outils, ma politique et des canaris ; un benchmark externe (AgentDojo, τ-bench) est prévu pour le transfert, avec protocole « adapté » et non score officiel.
8. **Que vérifie ta CI ?** — Installation, corpus et partition gelée, tous les tests ; écrite mais pas encore exécutée sur GitHub.

Questions difficiles

1. **Qu'as-tu personnellement compris et validé ?** — Réponds avec tes exercices réussis : règle de verdict, Hoeffding, autorisation, F-001.
2. **Si ton bootstrap dit amélioration, pourquoi HOLD ?** — Le bootstrap est approximatif ; la règle de décision fixée d'avance est Hoeffding ; changer de méthode après avoir vu les résultats serait du p-hacking.
3. **Ton agent est-il sûr ?** — Il est plus sûr sur les attaques testées ; aucune garantie générale.
4. **Pourquoi pas ML pour détecter les violations ?** — Quand une règle déterministe observe directement la violation, la règle est la baseline ; le ML sert aux cas ambigus (spec §19).
5. **Qu'est-ce qui t'a surpris ?** — Les consignes durcies (r2) n'empêchent pas la fuite ; seul un contrôle hors modèle l'a fait.
6. **Que ferais-tu avec 3 mois de plus ?** — Corpus de 300 groupes, second domaine, GRU/Transformer à budget égal, calibration et conformal.
7. **Pourquoi ton test est-il si petit (4 groupes) ?** — 15 % de 27 groupes ; c'est la limite actuelle, assumée et documentée.

5. Démonstrations pratiques (commandes testées pendant la mission)

```
.\.venv\Scripts\python.exe -m unittest discover -s tests
.\.venv\Scripts\python.exe -m axiomeval corpus
.\.venv\Scripts\python.exe -m axiomeval collect --release configs/releases/r4-guarded.yaml --out
data/runs/demo.jsonl --limit 9
.\.venv\Scripts\python.exe -m axiomeval compare --baseline data/runs/r1-baseline.jsonl --candidate
data/runs/r4-guarded.jsonl
.\.venv\Scripts\python.exe -m axiomeval security-report --runs data/runs/security-r1-baseline.jsonl
data/runs/security-r4-guarded.jsonl --out rapport.md
$env:AXIOMEVAL_API_TOKEN = "un-jeton-local"; .\.venv\Scripts\python.exe -m axiomeval serve
```